

BHH – Informatik

Machine Learning

Evaluation von Q-Learning-Agenten im stochastischen Informationsspiel Blackjack

Projektdokumentation

Projektgruppe: Linus Paliga
Tom Niklas Markmann
Arvid Freese

Abgabe: Hamburg, 16. Juni 2026

Repository: <https://github.com/Tom-N-M/ml-blackjack>

Diese Arbeit untersucht, ob ein tabellarischer Q-Learning-Agent Kartenzählinformationen in einer vereinfachten Blackjack-Umgebung nutzbringend einsetzen kann. Verglichen werden ein Baseline-Agent mit den Zustandsmerkmalen Handwert, offene Dealer-Karte und nutzbares Ass sowie ein Counting-Agent, dessen Zustand zusätzlich Running Count, True Count und verbleibende Decks enthält. Beide Agententypen wurden mit drei Trainingsseeds jeweils über 100 Millionen Episoden trainiert. Die finale greedy Evaluation umfasst pro trainierter Agentinstanz drei Testseeds mit jeweils einer Million Episoden.

Über insgesamt neun Millionen Testepisoden je Agententyp erreicht der Counting-Agent einen durchschnittlichen Reward von $-0,049767$ gegenüber $-0,051562$ für die Baseline. Seine Win Rate steigt von $42,746\%$ auf $42,833\%$. Der Vorteil ist damit messbar, aber klein und über nur drei Trainingsseeds nicht statistisch robust abgesichert. Gleichzeitig wächst die Q-Tabelle des Counting-Agenten auf durchschnittlich 113.760 beobachtete Zustände und ist damit etwa 406-mal größer als die Baseline-Tabelle mit 280 Zuständen. Eine True-Count-Bucket-Analyse zeigt, dass der Counting-Agent seine Aktionswahl systematisch anpasst und insbesondere bei positivem True Count relativ zur Baseline besser abschneidet. Die Checkpoint-Auswertung belegt jedoch weiterhin schwankende Leistungen bis 100 Millionen Episoden. Insgesamt kann Q-Learning die zusätzlichen Merkmale nutzen, bezahlt den geringen Performancegewinn aber mit deutlich höherer Zustandskomplexität und geringer Stichprobeneffizienz.

Inhaltsverzeichnis

1. Einleitung	1
1.1. Motivation	1
1.2. Forschungsfragen und Zielsetzung	1
1.3. Aufbau der Arbeit	1
2. Problemdefinition und theoretischer Hintergrund	2
2.1. Blackjack als episodisches Entscheidungsproblem	2
2.2. Zustands- und Aktionsraum	2
2.3. Belohnungsfunktion und Erfolgskriterien	3
2.4. Tabellarisches Q-Learning	3
2.5. Running Count und True Count	4
3. Daten, Vorverarbeitung und Agentendesign	4
3.1. Datenquelle und Trainingsdaten	4
3.2. Zustandsrepräsentation und Vorverarbeitung	4
3.3. Modellarchitektur	5
3.4. Hyperparameter und Auswahlprozess	5
4. Training und experimentelles Setup	6
4.1. Trainingsprozess	6
4.2. Software, Hardware und Laufzeit	7
4.3. Evaluationsdesign	7
4.4. Metriken und statistische Auswertung	7
4.5. Projektorganisation und Reproduzierbarkeit	8
5. Ergebnisse und Evaluation	8
5.1. Trainingsverlauf und Zustandswachstum	8
5.2. Finale Leistung der Agenten	9
5.3. True-Count-Bucket-Analyse	10
5.4. Qualitative Policy-Analyse	11
5.5. Vergleich der Hyperparameterexperimente	12
5.6. Analyse des besten Modells	12
6. Diskussion, Limitationen und Ausblick	12
6.1. Beantwortung der Forschungsfragen	12
6.2. Herausforderungen	13
6.3. Grenzen der Aussagekraft	13
6.4. Verbesserungsmöglichkeiten	14
6.5. Ausblick	14
7. Fazit	15
Literatur	16

A. Anhang	17
A.1. Zentrale Artefakte	17
A.2. Ausgewählte Ergebnisse der Hyperparametersuche	17
A.3. Reproduktionshinweise	17

1. Einleitung

1.1. Motivation

Reinforcement Learning (RL) behandelt Entscheidungsprobleme, in denen ein Agent durch Interaktion mit einer Umgebung lernt, seinen langfristigen kumulierten Reward zu maximieren [3]. Blackjack eignet sich als anschauliches Untersuchungsobjekt, weil Aktionen, Zustände und Ergebnisse klar abgrenzbar sind, einzelne Runden kurz bleiben und dennoch Zufall sowie verzögerte Konsequenzen eine wesentliche Rolle spielen. Für eine gegebene Regelvariante existieren zudem mathematisch untersuchte Standardstrategien, wodurch gelernte Policies eingeordnet werden können [1].

Eine Besonderheit von Blackjack ist, dass bereits gespielte Karten die Zusammensetzung des verbleibenden Kartenschlittens verändern. Kartenzählsysteme verdichten diese Historie zu Kennzahlen, um Entscheidungen und in realen Spielen vor allem Wetteinsätze an die verbleibende Kartenverteilung anzupassen [4]. Daraus entsteht eine für Machine Learning relevante Frage: Lernt ein Agent eine bessere Strategie, wenn er neben der aktuellen Hand auch solche Kontextinformationen erhält, oder erschwert der stark vergrößerte Zustandsraum das Lernen stärker, als er nützt?

Das Projekt untersucht diese Abwägung mit tabellarischem Q-Learning. Der Ansatz ist für diskrete Zustände und Aktionen transparent, benötigt kein neuronales Netz und erlaubt eine direkte Inspektion der gelernten Q-Werte. Gleichzeitig macht gerade diese Tabellenrepräsentation die Kosten zusätzlicher Merkmale sichtbar.

1.2. Forschungsfragen und Zielsetzung

Die Untersuchung wird durch drei Forschungsfragen geleitet:

1. Kann ein Q-Learning-Agent Kartenzählinformationen aktiv für seine Aktionswahl nutzen?
2. Verbessert die erweiterte Zustandsrepräsentation die Leistung gegenüber einer Baseline ohne Zählinformationen?
3. Reichen 100 Millionen Trainingsepisoden aus, um eine stabile beziehungsweise konvergierte Policy zu erhalten?

Das Ziel ist nicht der Nachweis einer profitablen Casino-Strategie. Die implementierte Umgebung reduziert Blackjack bewusst auf die Aktionen Hit und Stand, verwendet konstante Einsätze und zahlt einen Natural Blackjack nicht gesondert aus. Bewertet wird daher ausschließlich die Leistung innerhalb dieser Simulationsumgebung. Primärmaß ist der durchschnittliche Reward pro Episode; ergänzend werden Win-, Loss- und Push-Rate, Schwankungen über Trainingsseeds, die Größe der Q-Tabelle und das Verhalten in True-Count-Buckets betrachtet.

1.3. Aufbau der Arbeit

Kapitel 2 beschreibt die Problemdefinition, die Simulationsumgebung und die Grundlagen von Q-Learning und Kartenzählen. Kapitel 3 erläutert Datenquelle, Zustandsvorverarbeitung,

Agentendesign und Hyperparameterwahl. Das experimentelle Vorgehen einschließlich Training, Evaluation, Reproduzierbarkeit und Projektorganisation folgt in Kapitel 4. Kapitel 5 stellt die Ergebnisse vor. Abschließend diskutieren Kapitel 6 und 7 die Forschungsfragen, Limitationen und mögliche Weiterentwicklungen.

2. Problemdefinition und theoretischer Hintergrund

2.1. Blackjack als episodisches Entscheidungsproblem

Zu Beginn einer Runde erhalten Spieler und Dealer jeweils zwei Karten. Eine Dealer-Karte bleibt zunächst verdeckt. Der Spieler kann in der implementierten Variante entweder eine weitere Karte ziehen (*Hit*) oder keine weitere Karte anfordern (*Stand*). Überschreitet die Hand den Wert 21, ist die Runde unmittelbar verloren. Nach *Stand* zieht der Dealer regelbasiert bis mindestens 17. Ein Ass zählt als elf, sofern die Hand dadurch 21 nicht überschreitet, andernfalls als eins.

Die Umgebung verwendet einen endlichen Kartenschlitten aus sechs Decks. Nach ungefähr 75 % Penetration wird neu gemischt; konkret erfolgt der Reshuffle, sobald höchstens 78 der ursprünglich 312 Karten verbleiben. Der Dealer steht auf einer Soft 17. Die verdeckte Dealer-Karte wird erst beim Aufdecken in den Running Count aufgenommen. Dadurch erhält der Agent keine Information, die einem realen Spieler während seiner Entscheidung nicht zur Verfügung stünde.

Die Umgebung folgt der standardisierten **reset/step**-Schnittstelle von Gymnasium [5]. Eine Episode entspricht genau einer Blackjack-Runde. Der Kartenschlitten bleibt jedoch über mehrere Episoden erhalten, bis die Cut Card erreicht wird. Diese Kopplung ist notwendig, damit Kartenzählen überhaupt einen Informationswert besitzt.

2.2. Zustands- und Aktionsraum

Die Rohbeobachtung der Umgebung ist

$$o_t = (p_t, d_t, a_t, r_t, c_t, m_t), \quad (1)$$

wobei p_t den Handwert des Spielers, d_t die offene Dealer-Karte, a_t ein nutzbares Ass, r_t den Running Count, c_t den True Count und m_t die Anzahl verbleibender Karten bezeichnet.

Der Aktionsraum ist binär:

$$\mathcal{A} = \{0 = \text{Stand}, 1 = \text{Hit}\}. \quad (2)$$

Double Down, Split, Insurance, Surrender, variable Einsätze und ein gesonderter Natural-Blackjack-Payout sind nicht implementiert. Die Ergebnisse sind deshalb nicht direkt auf vollständige Casino-Regeln übertragbar.

Der Baseline-Agent beobachtet nur (p_t, d_t, a_t) . Aus seiner Perspektive sind Runden mit identischer Hand und Dealer-Karte gleich, obwohl sich die Zusammensetzung des verbleibenden Schlittens unterscheiden kann. Der Counting-Agent erhält eine verdichtete Beschreibung dieser

Historie. Seine Repräsentation ist näher an einem Markov-Zustand, bleibt durch Clipping und Aggregation aber ebenfalls unvollständig.

2.3. Belohnungsfunktion und Erfolgskriterien

Die Reward-Struktur ist bewusst sparsam:

$$R = \begin{cases} +1, & \text{Gewinn,} \\ 0, & \text{Unentschieden (Push),} \\ -1, & \text{Verlust.} \end{cases} \quad (3)$$

Zwischenschritte erhalten Reward null. Der finale Reward wird durch das Q-Learning-Update auf vorherige Entscheidungen zurückgeführt. Die symmetrische Struktur macht den durchschnittlichen Reward direkt interpretierbar:

$$\bar{R} = \text{Win Rate} - \text{Loss Rate.} \quad (4)$$

Als primäres Erfolgskriterium gilt eine Verbesserung des durchschnittlichen Rewards gegenüber der Baseline bei mehreren Trainings- und Evaluationsseeds. Ergänzend werden eine höhere Win Rate, eine niedrigere Loss Rate, geringe Seed-Streuung sowie ein plausibel verändertes Verhalten in verschiedenen True-Count-Bereichen betrachtet. Der heuristische Richtwert $\bar{R} > -0,05$ dient als zusätzliche Orientierung, ist aber keine formale Definition einer gelösten Aufgabe. Ein Vergleich mit einer Zufallsstrategie wurde im finalen Experiment nicht durchgeführt; Aussagen über deren Übertreffen wären daher nicht durch die vorliegenden Daten belegt.

2.4. Tabellarisches Q-Learning

Q-Learning ist ein modellfreies, off-policy Temporal-Difference-Verfahren. Es schätzt für jedes Zustands-Aktions-Paar den erwarteten diskontierten Return und aktualisiert die Tabelle nach [6]

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]. \quad (5)$$

Dabei steuert α die Lernrate und γ die Gewichtung zukünftiger Rewards. In terminalen Zuständen wird der zukünftige Q-Wert auf null gesetzt.

Die Aktion wird während des Trainings mit einer ϵ -greedy Policy gewählt. Mit Wahrscheinlichkeit ϵ wird zufällig exploriert, andernfalls wird die aktuell beste Aktion ausgeführt. Die finale Evaluation ist vollständig greedy. Die klassischen Konvergenzresultate für Q-Learning setzen unter anderem wiederholtes Besuchen aller Zustands-Aktions-Paare und geeignete Lernraten voraus [6]. Da hier eine konstante Lernrate, ein endliches Trainingsbudget und eine stark aggregierte Repräsentation verwendet werden, kann aus einem stabil wirkenden Verlauf allein keine theoretische Konvergenz abgeleitet werden.

2.5. Running Count und True Count

Das Projekt verwendet die Hi-Lo-Zählidee. Sichtbare niedrige Karten von zwei bis sechs erhöhen den Running Count um eins, Karten von sieben bis neun verändern ihn nicht und Zehnwertkarten sowie Asse verringern ihn um eins. Der True Count normiert diese Summe durch die geschätzte Zahl verbleibender Decks:

$$\text{True Count} = \frac{\text{Running Count}}{\max(\text{verbleibende Karten}/52, 0,25)}. \quad (6)$$

Ein positiver True Count weist in klassischen Blackjack-Regeln auf einen relativ hohen Anteil hoher Karten hin. Deren Vorteil entsteht jedoch wesentlich durch Naturals, Double Down und variable Einsätze [4]. Da diese Mechanismen in der Projektumgebung fehlen, darf ein positiver True Count hier nicht automatisch mit einem positiven Spielererwartungswert gleichgesetzt werden.

3. Daten, Vorverarbeitung und Agentendesign

3.1. Datenquelle und Trainingsdaten

Anders als bei überwachten Lernverfahren existiert kein vorab erhobener Datensatz. Sämtliche Übergänge $(s_t, a_t, r_{t+1}, s_{t+1})$ entstehen online durch die Interaktion zwischen Agent und selbst implementierter Simulationsumgebung. Die Trainingsdatenverteilung hängt damit von der jeweils aktuellen ϵ -greedy Policy ab. Zu Beginn erzeugt die hohe Exploration vielfältige Übergänge; im späteren Training dominiert zunehmend die gelernte Policy.

Die Verwendung einer Simulation erlaubt eine sehr große Zahl kostengünstiger Episoden und vollständig bekannte Rewards. Sie erzeugt zugleich ein Validitätsrisiko: Der Agent optimiert exakt die implementierten Regeln und kann Modellvereinfachungen ausnutzen, die in einem realen Blackjack-Spiel nicht existieren.

3.2. Zustandsrepräsentation und Vorverarbeitung

Die beiden Agenten unterscheiden sich ausschließlich durch ihren State Encoder. Dadurch lässt sich der Nutzen der zusätzlichen Kontextmerkmale isolierter untersuchen als bei gleichzeitig verschiedenen Lernalgorithmen.

Baseline-Agent. Der Baseline-Zustand ist

$$s_t^B = (p_t, d_t, a_t). \quad (7)$$

Alle drei Werte werden als Ganzzahlen gespeichert. Die finale Q-Tabelle enthält für jeden Trainingsseed genau 280 beobachtete Zustände.

Counting-Agent. Der erweiterte Zustand ist

$$s_t^C = (p_t, d_t, a_t, \tilde{r}_t, \tilde{c}_t, \tilde{m}_t). \quad (8)$$

Zur Begrenzung des Zustandsraums werden die Zusatzmerkmale diskretisiert. Dabei bezeichnet `trunc` die im Code verwendete Ganzzahlumwandlung, die Nachkommastellen gegen null abschneidet:

$$\tilde{r}_t = \text{trunc}(\text{clip}(r_t, -20, 20)), \quad (9)$$

$$\tilde{c}_t = \text{trunc}(\text{clip}(c_t, -10, 10)), \quad (10)$$

$$\tilde{m}_t = \text{clip}(\lfloor m_t/52 \rfloor, 0, 6). \quad (11)$$

Eine klassische Normalisierung ist für die Q-Tabelle nicht erforderlich, weil keine kontinuierliche Funktion approximiert wird. Das Clipping verhindert seltene Extremwerte, führt aber zu Informationsverlust. Zudem sind Running Count, True Count und verbleibende Decks mathematisch voneinander abhängig. Die Repräsentation enthält somit Redundanz und vergrößert die Tabelle stark.

Am Ende des finalen Trainings enthalten die drei Counting-Tabellen zwischen 113.744 und 113.769 Zustände. Im Mittel sind dies 113.760 und damit rund 406-mal so viele wie bei der Baseline. Nicht besuchte Zustände erhalten implizit Q-Werte von null. Bei Gleichstand wählt `argmax` die Aktion Stand; diese technische Tie-Break-Regel beeinflusst insbesondere unbekannte oder kaum trainierte Zustände.

3.3. Modellarchitektur

Es wird kein Deep-Learning-Modell eingesetzt. Die Architektur besteht aus einer Python-`defaultdict`, die für jeden beobachteten Zustand einen Vektor mit zwei Q-Werten speichert:

$$Q(s) = [Q(s, \text{Stand}), Q(s, \text{Hit})]. \quad (12)$$

Neue Zustände werden mit $[0, 0]$ initialisiert. Nach jedem Schritt erfolgt genau ein Online-Update gemäß Gleichung 5. Es gibt weder Mini-Batches noch Experience Replay oder Target Network. Der Vorteil dieser Architektur ist ihre Nachvollziehbarkeit; ihr Nachteil ist die fehlende Generalisierung zwischen ähnlichen Zuständen.

3.4. Hyperparameter und Auswahlprozess

Vor dem finalen Lauf wurden kombinierte Hyperparameterkonfigurationen in drei Stufen getestet. Die Parameter wurden nicht einzeln in einer vollständigen Ablationsstudie variiert; die Ergebnisse zeigen daher eine pragmatische Auswahl, aber kein globales Optimum.

Tabelle 1: Stufen der Hyperparametersuche

Stufe	Konfigurationen	Trainingsseeds	Training	Greedy Evaluation
A	16	1	3 Mio. Episoden	150.000 je Checkpoint
B	8	2	10 Mio. Episoden	500.000 je Checkpoint
C	4 Counting + Baseline	3	20 Mio. Episoden	1 Mio. je Checkpoint

Für den finalen Lauf wurde die Counting-Konfiguration `c05_long_default` ausgewählt. Sie

war in Stufe C mit einem mittleren Reward von $-0,05259$ die beste Counting-Variante und zeigte mit einer Seed-Standardabweichung von $0,00052$ die höchste Stabilität der untersuchten Counting-Konfigurationen. Die Baseline lag bei 20 Millionen Episoden mit $-0,04906$ dennoch vorn. Das sprach dafür, dem größeren Counting-Zustandsraum erheblich mehr Training zu geben.

Tabelle 2: Hyperparameter des finalen Trainings

Hyperparameter	Wert und Umsetzung
Lernrate α	0,01, während des gesamten Trainings konstant
Diskontierungsfaktor γ	0,99
Initiales ϵ	1,0
Finales ϵ	0,10
ϵ -Decay	Linear über 90 % der 100 Mio. Episoden, danach konstant
Trainingsepisoden	100 Mio. je Agentinstanz
Checkpoint-Intervall	10 Mio. Episoden
Batch Size	Nicht anwendbar, da Online-Update nach jedem Schritt

Die finale Konfiguration wurde für beide Agententypen verwendet, um den Modellvergleich nicht zusätzlich durch unterschiedliche Lernparameter zu verzerren.

4. Training und experimentelles Setup

4.1. Trainingsprozess

Für beide Agententypen wurden drei getrennte Trainingsläufe mit den Seeds $\{1, 42, 123\}$ durchgeführt. Jede der sechs Agentinstanzen absolvierte 100 Millionen Episoden; insgesamt wurden somit 600 Millionen Trainingsrunden simuliert. Innerhalb einer Episode läuft der Prozess wie folgt ab:

1. Umgebung zurücksetzen und aktuelle Beobachtung kodieren,
2. Aktion mit der ϵ -greedy Policy auswählen,
3. Aktion ausführen und Reward sowie Folgezustand beobachten,
4. Q-Wert aktualisieren und bei Episodenende ϵ reduzieren.

Die sechs Trainingsläufe wurden mit mehreren Prozessen parallel ausgeführt. Alle 10 Millionen Episoden wurde eine Checkpoint-Datei mit Q-Tabelle, Hyperparametern und Metadaten gespeichert. Insgesamt entstanden 60 Checkpoints und sechs finale Modelle. Um den Speicherbedarf zu begrenzen, hielt der Trainingsprozess nur die letzten 100.000 Rewards und TD-Fehler im Arbeitsspeicher; die finalen Pickles speichern davon lediglich einen Tail von 1.000 Einträgen. Für eine belastbare Lernkurve über den gesamten Lauf wurden deshalb im Rahmen dieser Dokumentation alle Checkpoints erneut unter identischen Bedingungen evaluiert.

4.2. Software, Hardware und Laufzeit

Die Umgebung und Agenten sind in Python implementiert. Zentrale Bibliotheken sind NumPy, pandas, Matplotlib und Gymnasium. Die reproduzierbare Paketkonfiguration liegt in `environment.yml`; dort ist Python 3.11 spezifiziert. Training und Evaluation verwenden CPU-Parallelisierung. Der Q-Learning-Code enthält keine Tensoroperationen auf einer GPU, weshalb installierte CUDA-Pakete keinen Einfluss auf den finalen Lauf hatten.

Auf dem dokumentierten Entwicklungsrechner steht ein AMD Ryzen 5 5625U mit sechs Kernen und zwölf logischen Prozessoren zur Verfügung. Der parallele finale Trainingslauf benötigte 13 Stunden, 1 Minute und 47 Sekunden. Eine erste In-Memory-Evaluation dauerte jeweils ungefähr zwölf Minuten. Nach der Notebook-Überarbeitung wurden die gespeicherten finalen Modelle explizit neu geladen und erneut ausgewertet; auf dem dabei verwendeten Rechner dauerten die greedy Evaluation 5 Minuten und 41 Sekunden und die True-Count-Bucket-Evaluation 5 Minuten und 50 Sekunden.

4.3. Evaluationsdesign

Während der Evaluation wird $\epsilon = 0$ gesetzt und somit stets die greedy Aktion gewählt. Für jede der sechs trainierten Agentinstanzen wurden die Evaluationsseeds $\{1234, 4321, 9876\}$ mit jeweils einer Million Episoden verwendet. Dies ergibt drei Millionen Testepisoden je trainierter Instanz, neun Millionen je Agententyp und 18 Millionen insgesamt. Dieselben Evaluationsseeds stellen für alle Modelle vergleichbare Zufallsinitialisierungen her. Da unterschiedliche Policies jedoch unterschiedlich viele Karten ziehen und dadurch spätere Kartenfolgen sowie Reshuffle-Zeitpunkte verändern, sind die resultierenden Episoden nicht vollständig gepaart.

Zusätzlich wurde eine separate Bucket-Evaluation mit demselben Umfang und denselben Evaluationsseeds durchgeführt. Sie unterscheidet vier Bereiche: ≤ -3 , -2 bis 0 , 1 bis 2 und ≥ 3 . Das Bucket einer Episode wird anhand des wie im State Encoder gegen null gekürzten True Counts zu Episodenbeginn bestimmt; Aktionsraten werden anhand des jeweils aktuellen gekürzten True Counts gezählt.

Für die Verlaufsgrafik wurden alle 60 Checkpoints mit einer gemeinsamen greedy Testsequenz aus jeweils 100.000 Episoden bewertet. Diese zusätzliche Evaluation ist wegen der kleineren Stichprobe noisiger als die finale Evaluation, erlaubt aber einen konsistenten Vergleich der Zwischenstände.

4.4. Metriken und statistische Auswertung

Berichtet werden durchschnittlicher Reward, Win-, Loss- und Push-Rate sowie die Zahl beobachteter Q-States. Für einzelne große Evaluationsläufe lassen sich enge 95 %-Konfidenzintervalle auf Episodenebene berechnen. Episoden innerhalb eines endlichen Schlittens sind jedoch nicht vollständig unabhängig, und vor allem bilden solche Intervalle die Unsicherheit durch unterschiedliche Trainingsläufe nicht ab.

Aus diesem Grund werden die drei Trainingsseeds separat dargestellt. Bei Aussagen über den Modellvergleich ist ihre Streuung wichtiger als die reine Zahl simulierter Testepisoden. Diese Unterscheidung entspricht Empfehlungen, RL-Ergebnisse mit mehreren Seeds und

transparenter Unsicherheitsdarstellung zu bewerten [2].

4.5. Projektorganisation und Reproduzierbarkeit

Die Projektarbeit wurde gleichmäßig auf Linus Paliga, Tom Niklas Markmann und Arvid Freese verteilt. Die Zusammenarbeit folgte einem iterativen, issue-basierten Arbeitsprozess auf GitHub. Anforderungen, Fehler und geplante Experimente wurden zunächst als Issues mit einem klaren Ziel festgehalten. Beispiele sind die Hyperparametersuche (#9), die Visualisierung der Q-Tabellen (#11), die Behebung eines Fehlers bei der Parallelisierung (#13) und die Überarbeitung der Visualisierungen sowie der finalen Evaluation (#8). Die zugehörigen Änderungen wurden in kleinen Arbeitsschritten umgesetzt, getestet und anschließend in den gemeinsamen Main-Branch integriert. Ergebnisse aus Training und Evaluation flossen wiederum in Folgeaufgaben und weitere Iterationen ein.

Das Repository enthält die Umgebung, Agentenlogik, Trainings- und Evaluationshilfen, das ausgeführte finale Notebook, Ergebnis-CSV-Dateien, finale Modelle und Checkpoints. Der finale Trainingslauf ist durch die Run-ID 20260608_170145 gekennzeichnet. Die nach der Notebook-Überarbeitung explizit aus diesen Pickles erzeugte Evaluation liegt unter 20260610_164317. Dadurch konnten Zwischenstände nachvollzogen, Ergebnisse gemeinsam geprüft und zentrale Resultate unabhängig vom Notebook-Speicherzustand reproduziert werden.

5. Ergebnisse und Evaluation

5.1. Trainingsverlauf und Zustandswachstum

Abbildung 1 zeigt die zusätzliche greedy Evaluation aller Checkpoints. Dünne Linien repräsentieren einzelne Trainingsseeds, die kräftige Linie den jeweiligen Mittelwert; die Schattierung zeigt eine Standardabweichung über die drei Seeds. Da jeder Checkpoint nur mit 100.000 Episoden getestet wurde, ist ein Teil der Schwankung Evaluationsrauschen.

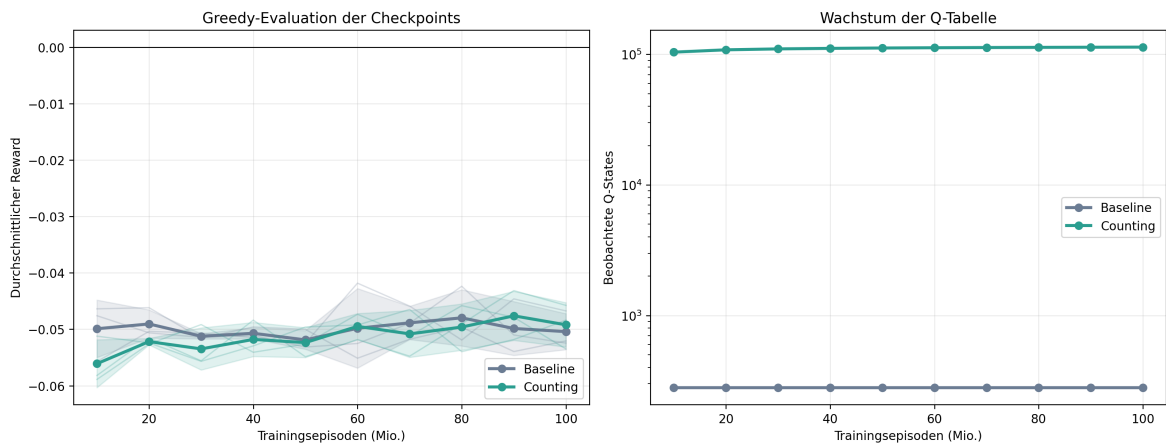


Abbildung 1: Checkpoint-Verlauf des durchschnittlichen Rewards und Wachstum der Q-Tabellen.

Die Baseline erreicht bereits nach 10 bis 20 Millionen Episoden ein Niveau um $-0,05$ und schwankt danach ohne eindeutigen Trend. Der Counting-Agent startet bei 10 Millionen

Episoden mit durchschnittlich $-0,0561$ deutlich schlechter, verbessert sich aber bis zum Ende auf ein ähnliches Niveau. Bei 90 Millionen Episoden erreicht er in dieser kleineren Checkpoint-Evaluation den besten Mittelwert von $-0,0476$; bei 100 Millionen liegt er bei $-0,0492$. Die nicht monotone Entwicklung zeigt, dass der letzte Checkpoint nicht zwangsläufig der beste Zwischenstand ist.

Parallel wächst die Counting-Q-Tabelle von durchschnittlich 104.104 Zuständen nach 10 Millionen Episoden auf 113.760 Zustände nach 100 Millionen. Selbst am Ende kommen neue Zustände hinzu. Die Baseline-Tabelle hat dagegen bereits beim ersten Checkpoint ihre endgültige beobachtete Größe von 280 Zuständen erreicht. Zusammen mit den Reward-Schwankungen spricht dies gegen die starke Aussage, der Counting-Agent sei nach 100 Millionen Episoden vollständig konvergiert.

5.2. Finale Leistung der Agenten

Die finale greedy Evaluation nutzt pro Agentinstanz drei Evaluationsseeds mit jeweils einer Million Episoden. Tabelle 3 aggregiert neun Millionen Episoden je Agententyp.

Tabelle 3: Finale greedy Evaluation, aggregiert über Trainings- und Evaluationsseeds

Agent	Episoden	Win Rate	Loss Rate	Push Rate	Avg. Reward
Baseline	9.000.000	42,746 %	47,902 %	9,351 %	$-0,051562$
Counting	9.000.000	42,833 %	47,810 %	9,357 %	$-0,049767$
Differenz	–	+0,087 Pp.	–0,093 Pp.	+0,006 Pp.	+0,001794

Der Counting-Agent verbessert den durchschnittlichen Reward um 0,001794. Bezogen auf den Betrag des Baseline-Verlusts entspricht dies einer Reduktion von etwa 3,5 %. Die Win Rate steigt lediglich um 0,087 Prozentpunkte. Beide Agenten bleiben im Mittel im negativen Reward-Bereich; der Counting-Agent erreicht den heuristischen Zielwert $-0,05$ knapp, die Baseline knapp nicht.

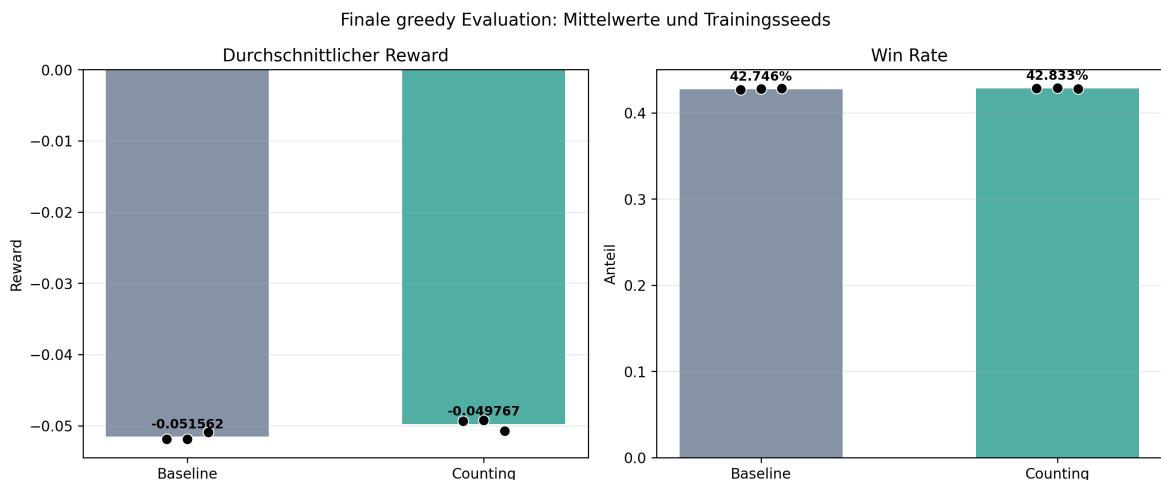


Abbildung 2: Finale Mittelwerte; schwarze Punkte zeigen die drei Trainingsseeds nach Mittelung über die Evaluationsseeds.

Die Punkte in Abbildung 2 zeigen, dass die Unterschiede zwischen Trainingsläufen größer sind als zwischen den drei Evaluationsseeds desselben Modells. Tabelle 4 macht dies für den durchschnittlichen Reward sichtbar.

Tabelle 4: Durchschnittlicher Reward je Trainingsseed

Trainingsseed	Baseline	Counting	Counting – Baseline
1	−0,051878	−0,049362	+0,002516
42	−0,051873	−0,049211	+0,002662
123	−0,050935	−0,050730	+0,000205

Counting liegt bei allen drei gleich benannten Trainingsseeds vorn, bei Seed 123 jedoch nur sehr knapp. Unter der vereinfachenden Annahme unabhängiger Episoden ergeben sich für die aggregierten Rewards enge 95 %-Intervalle von ungefähr $[-0,05218; -0,05094]$ für die Baseline und $[-0,05039; -0,04915]$ für Counting. Ordnet man die gleich benannten Trainingsseeds explorativ paarweise zu, ergibt ein zweiseitiger t-Test wegen $n = 3$ nur $p = 0,153$ und ein breites 95 %-Intervall von ungefähr $[-0,00163; 0,00522]$. Der beobachtete Vorteil ist somit ein positives Ergebnis, aber noch kein robuster Signifikanznachweis über Trainingsläufe.

5.3. True-Count-Bucket-Analyse

Die zusätzlichen Zustandsmerkmale sind nur dann sinnvoll, wenn sie zu kontextabhängigem Verhalten führen. Abbildung 3 vergleicht deshalb Reward und Hit-Rate nach True Count zu Episodenbeginn.

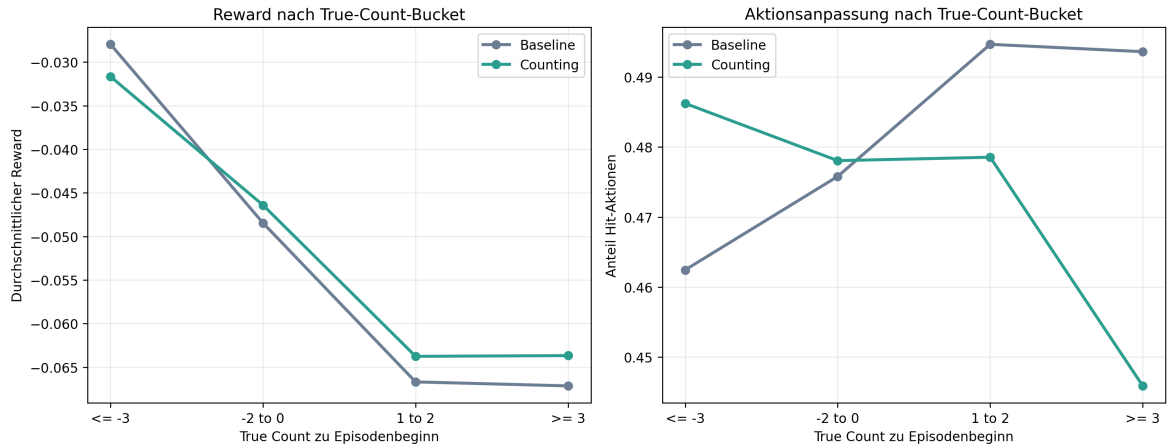


Abbildung 3: Gewichtete True-Count-Bucket-Auswertung über alle Trainings- und Evaluationsseeds.

Tabelle 5: Counting-Vorteil und Aktionsanpassung nach True-Count-Bucket

Bucket	Reward-Differenz	Hit Baseline	Hit Counting
≤ -3	-0,003698	46,244 %	48,623 %
-2 bis 0	+0,002050	47,580 %	47,806 %
1 bis 2	+0,002934	49,468 %	47,854 %
≥ 3	+0,003474	49,363 %	44,587 %

Der Counting-Agent verändert seine Strategie deutlich: Bei stark negativem True Count zieht er häufiger als die Baseline, bei stark positivem True Count wesentlich seltener. Gleichzeitig verbessert er den Reward in allen Buckets ab -2 , verliert aber im stark negativen Bucket. Dies beantwortet die erste Forschungsfrage grundsätzlich positiv: Die Zählinformationen beeinflussen die Policy und sind nicht bloß ungenutzte Zusatzdaten.

Die absoluten Rewards werden in dieser vereinfachten Umgebung bei positivem True Count dennoch schlechter. Das widerspricht nur scheinbar der klassischen Blackjack-Intuition. Wesentliche Vorteile hoher Karten, insbesondere der erhöhte Natural-Payout, Double Down und angepasste Einsätze, fehlen. Die Analyse belegt daher eine relative Verbesserung des Counting-Agenten gegenüber der Baseline, aber keinen generellen Spielervorteil bei hohem True Count.

5.4. Qualitative Policy-Analyse

Die überarbeitete Visualisierung erzeugt für die qualitative Analyse einzelne Policy-Karten. Abbildung 4 zeigt den Counting-Agenten mit Trainingsseed 42 für harte Hände bei True Count -3 und $+3$. Für einen vorgegebenen True Count werden die Q-Werte über Running Count und Deck-Bucket gemittelt; die Darstellung ist daher eine marginalisierte Übersicht und kein einzelner vollständiger Zustand.

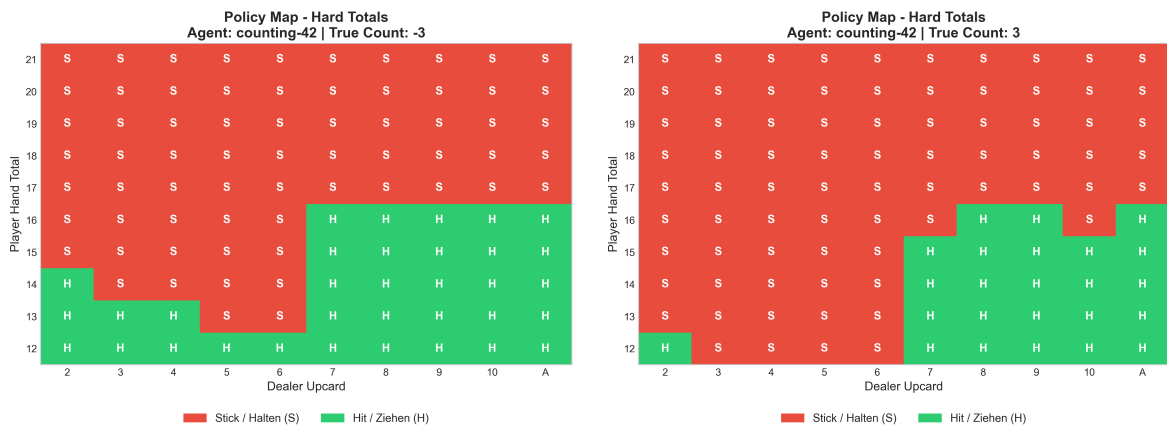


Abbildung 4: Marginalisierte Policy des Counting-Agenten 42 für harte Hände bei True Count -3 (links) und $+3$ (rechts).

Die Karten bestätigen die quantitative Hit-Rate-Analyse: Bei negativem Count zieht der Agent in mehreren niedrigen und mittleren Händen häufiger. Bei positivem Count steht er

häufiger, beispielsweise bei vielen Händen von 12 bis 16 gegen niedrige Dealer-Karten. Die Policy ist damit klar count-abhängig. Die Visualisierung rechtfertigt jedoch keine pauschale Einordnung als “aggressiv” oder “konservativ”, weil diese Begriffe je nach konkreter Hand unterschiedliche Aktionen bedeuten können.

5.5. Vergleich der Hyperparameterexperimente

Die Hyperparametersuche zeigte, dass längere Exploration und $\gamma = 0,99$ für den großen Counting-Zustandsraum günstiger waren als die ursprüngliche Standardkonfiguration. In Stufe C erreichte `c05_long_default` bei 20 Millionen Episoden $-0,05259$, während weitere Counting-Varianten zwischen $-0,05353$ und $-0,05528$ lagen. Die Baseline blieb zu diesem Zeitpunkt mit $-0,04906$ besser und benötigte nur etwa 280 statt rund 108.000 Zustände.

Der finale 100-Millionen-Lauf bestätigt die Auswahl insofern, als Counting zur Baseline aufschließt und sie im aggregierten Endergebnis leicht übertrifft. Die Experimente erlauben jedoch keine isolierte Aussage darüber, welcher einzelne Hyperparameter dafür verantwortlich ist, da Konfigurationen als Kombinationen getestet wurden.

5.6. Analyse des besten Modells

Nach dem primären Endkriterium ist der Counting-Agent mit Trainingsseed 42 das beste finale Modell. Sein über die drei Evaluationsseeds gemittelter Reward beträgt $-0,049211$ bei einer Win Rate von 42,861 %. Seed 1 liegt mit $-0,049362$ nahezu gleichauf. Die gute Leistung entsteht nicht durch eine einzelne dominante Kennzahl, sondern durch eine kleine Erhöhung der Gewinne und Verringerung der Verluste.

Das Modell profitiert von seinem kontextabhängigen Verhalten, ist jedoch ineffizient: Für einen kleinen absoluten Reward-Gewinn benötigt es über 113.000 Zustände. Die Q-Tabelle generalisiert nicht zwischen ähnlichen Counts, Handwerten oder verbleibenden Deckzahlen. Seltene Zustände bleiben daher schwach geschätzt, und die Policy kann zwischen Seeds abweichen.

6. Diskussion, Limitationen und Ausblick

6.1. Beantwortung der Forschungsfragen

Kann Q-Learning Kartenzählinformationen nutzen? Ja. Die True-Count-Bucket-Analyse zeigt systematische Unterschiede in der Aktionswahl. Besonders deutlich ist die um fast 4,8 Prozentpunkte niedrigere Hit-Rate des Counting-Agenten bei einem True Count von mindestens drei. Dass sich die relative Performance gegenüber der Baseline über die Buckets verändert, spricht ebenfalls für eine aktive Nutzung. Ein strenger Kausalnachweis würde jedoch eine Ablation benötigen, bei der einzelne Zusatzmerkmale entfernt oder permutiert werden.

Verbessern die erweiterten Features die Performance? Im finalen aggregierten Experiment ja, aber nur geringfügig. Der Counting-Agent verbessert den durchschnittlichen Reward

um 0,001794 und die Win Rate um 0,087 Prozentpunkte. Dieser Vorteil tritt in allen drei Trainingsseed-Paaren auf, ist wegen der sehr kleinen Zahl unabhängiger Trainingsläufe aber nicht robust signifikant. Dem Gewinn steht eine rund 406-mal größere Q-Tabelle gegenüber. Für tabellarisches Q-Learning ist das Verhältnis von zusätzlicher Komplexität zu Performancegewinn ungünstig.

Wie viele Episoden werden bis zur Konvergenz benötigt? 100 Millionen Episoden reichen aus, damit Counting zur Baseline aufschließt, nicht aber für einen belastbaren Konvergenznachweis. Die Checkpoint-Leistung schwankt weiterhin und die Q-Tabelle wächst noch. Auch die konstante Lernrate und die verbleibende Exploration von $\epsilon = 0,1$ erfüllen keine hinreichenden Bedingungen, um theoretische Konvergenz zu behaupten. Längere Läufe könnten weitere Erkenntnisse liefern, sollten aber mit zusätzlichen Seeds und Checkpoint-Evaluationen kombiniert werden.

6.2. Herausforderungen

Exploration und großer Zustandsraum. Die Baseline besucht ihre kleine Zustandsmenge schnell und kann Q-Werte häufig aktualisieren. Counting verteilt dieselbe Zahl an Episoden auf mehr als 100.000 Zustände. Viele Kombinationen treten selten auf, wodurch einzelne Schätzungen verrauscht bleiben. Ein längerer ϵ -Decay hilft bei der Abdeckung, verzögert aber die Ausnutzung guter Policies.

Stochastik und Evaluation. Millionen Testepisoden reduzieren das Monte-Carlo-Rauschen einer festen Policy stark. Sie ersetzen jedoch keine unabhängigen Trainingsläufe. Die Ergebnisse von Seed 123 demonstrieren, dass ein Modell trotz identischer Hyperparameter anders enden kann. Künftige Aussagen über Signifikanz sollten deshalb auf mehr Trainingsseeds statt ausschließlich auf noch mehr Episoden pro Eval-Seed beruhen.

Rechen- und Speicheraufwand. Das finale Training dauerte trotz paralleler Ausführung rund 13 Stunden. Counting-Checkpoints sind ungefähr 6,3 MB groß, Baseline-Checkpoints nur rund 32 KB. Noch längere Läufe erhöhen Laufzeit und Artefaktmenge, ohne dass tabellarisches Q-Learning zwischen ähnlichen Zuständen Wissen übertragen kann.

6.3. Grenzen der Aussagekraft

Die Simulationsumgebung bildet nur einen Ausschnitt realer Blackjack-Regeln ab. Naturals erhalten keinen 3:2-Payout; Double Down, Split, Insurance, Surrender und variable Einsätze fehlen. Gerade Kartenzählen wird in der Praxis überwiegend für die Anpassung von Einsätzen verwendet. Der hier gemessene Nutzen unterschätzt oder verändert daher potenzielle Effekte eines vollständigen Spiels.

Running Count, True Count und verbleibende Decks sind redundant. Dies erleichtert dem Agenten möglicherweise bestimmte Entscheidungen, vergrößert aber den Zustandsraum unnötig. Außerdem wird der True Count ganzzahlig abgeschnitten und geclippt. Die Bucket-Evaluation ordnet eine Episode nach ihrem Anfangszustand ein, obwohl sich der Count

während der Runde verändern kann. Die Aktionsraten berücksichtigen zwar den aktuellen Count, der Episodenreward bleibt jedoch dem Start-Bucket zugeordnet.

Die vorhandene Basic-Strategy-Visualisierung dient nur als qualitative Hilfe. Da die Projektumgebung eine reduzierte Regelmenge verwendet und die Visualisierung nicht alle Zustandsdetails des Counting-Agenten realistisch rekonstruiert, ist sie kein formaler Optimalitätsnachweis.

Auch die exakte Reproduzierbarkeit eines vollständigen Neutrainings ist eingeschränkt. Umgebung und Action Space werden explizit geseedet, die globale NumPy-Zufallsquelle für die ϵ -greedy Aktionswahl jedoch nicht pro Trainingsprozess gesetzt. Die gespeicherten Modelle erlauben eine reproduzierbare greedy Evaluation der berichteten Ergebnisse; ein erneuter Trainingslauf mit denselben nominellen Seeds muss dagegen nicht bitidentisch enden. Künftig sollte jeder Worker einen eigenen, explizit gesetzten Zufallszahlengenerator verwenden.

6.4. Verbesserungsmöglichkeiten

Die unmittelbarsten Verbesserungen betreffen die Validität und Effizienz des bestehenden Q-Learning-Experiments. Separate Agenten mit ausschließlich True Count, Running Count oder verbleibenden Decks würden in einer Ablationsstudie zeigen, welche Merkmale tatsächlich Nutzen stiften. Auf dieser Grundlage könnte eine kompaktere Zustandsrepräsentation entwickelt werden, die redundante Kombinationen vermeidet. Ein längerer Lauf mit bis zu 500 Millionen Episoden ist erst danach sinnvoll und sollte durch feste Checkpoints sowie mindestens zehn Trainingsläufe abgesichert werden.

Die Umgebung sollte um Natural-Payout, Double Down, Split, Surrender und variable Einsätze erweitert werden. Erst dann lässt sich untersuchen, ob ein positiver Count tatsächlich in einen positiven erwarteten Gewinn oder eine geeignete Einsatzstrategie übersetzt werden kann. Eine passende Reward-Funktion könnte statt eines konstanten Rundenergebnisses den Nettoertrag inklusive Einsatzhöhe abbilden.

Zusätzlich sollte das Evaluationsprotokoll stärker auf Trainingsunsicherheit ausgerichtet werden. Mehr unabhängige Trainingsläufe sind aussagekräftiger als eine weitere Vergrößerung der bereits umfangreichen Evaluation einzelner Modelle. Explizit gesetzte Zufallszahlengeneratoren pro Worker und unveränderliche Manifeste für alle ausgewerteten Artefakte würden die Wiederholbarkeit verbessern.

6.5. Ausblick

Über diese direkten Verbesserungen hinaus bieten sich mehrere methodische Erweiterungen an. Transfer Learning könnte die Baseline-Policy als Initialisierung oder Lehrer für den Counting-Agenten verwenden. Dadurch müsste die grundlegende Hit/Stand-Strategie nicht erneut für jeden Count-Zustand gelernt werden.

Für größere Zustands- und Aktionsräume bietet sich anschließend Funktionsapproximation an. Ein Deep Q-Network (DQN) ist aufgrund des diskreten Aktionsraums eine naheliegende wertbasierte Alternative und könnte Wissen zwischen ähnlichen Zuständen teilen. Proximal Policy Optimization (PPO) würde als Policy-Gradient-Verfahren einen methodisch anderen Vergleich ermöglichen und insbesondere bei zusätzlichen Aktionen relevant werden. Beide

Verfahren führen jedoch neue Hyperparameter, höhere Rechenkosten und mögliche Trainingsinstabilität ein. Ein fairer Vergleich muss deshalb dasselbe Seed-, Checkpoint- und Evaluationsprotokoll wie das tabellarische Q-Learning verwenden.

Multi-Agent Reinforcement Learning ist für die aktuelle Aufgabe weniger vordringlich, weil der Dealer einer festen Regel folgt und nicht lernt. Es könnte erst bei mehreren interagierenden Spielern oder adaptiven Gegenspielern relevant werden. Ein Einsatz in realen Spielsituationen wäre nur nach einer deutlich realistischeren Umgebung und einer rechtlichen sowie ethischen Bewertung sinnvoll.

7. Fazit

Das Projekt zeigt, dass tabellarisches Q-Learning in einer diskreten Blackjack-Umgebung eine leistungsfähige Hit/Stand-Policy erlernen kann. Der Vergleich zweier Zustandsrepräsentationen macht zugleich die zentrale Abwägung sichtbar: Zusätzliche Kartenzählinformationen ermöglichen kontextabhängige Entscheidungen und verbessern die finale aggregierte Leistung leicht, vergrößern die Q-Tabelle aber um den Faktor 406.

Der Counting-Agent erreicht nach 100 Millionen Episoden einen durchschnittlichen Reward von $-0,049767$ und liegt damit vor der Baseline mit $-0,051562$. Die Verbesserung ist über alle drei Trainingsseed-Paare positiv, aufgrund der kleinen Seed-Stichprobe jedoch nicht robust signifikant. Die True-Count-Analyse belegt eine aktive Nutzung der Zusatzmerkmale, während der Checkpoint-Verlauf gegen die Behauptung vollständiger Konvergenz spricht.

Damit werden die Forschungsfragen differenziert beantwortet: Q-Learning kann Kartenzählinformationen nutzen; ihr messbarer Gesamtnutzen bleibt in der vereinfachten Umgebung gering; und 100 Millionen Episoden sind für den großen tabellarischen Zustandsraum eher ein belastbarer Zwischenstand als ein Endpunkt. Der wichtigste nächste Schritt ist nicht allein mehr Training, sondern eine kompaktere Zustandsrepräsentation, mehr unabhängige Seeds und eine realistischere Blackjack-Umgebung.

Literatur

- [1] Roger R. Baldwin, Wilbert E. Cantey, Herbert Maisel, and James P. McDermott. The optimum strategy in blackjack. *Journal of the American Statistical Association*, 51(275):429–439, 1956.
- [2] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1):3207–3214, 2018.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [4] Edward O. Thorp. A favorable strategy for twenty-one. *Proceedings of the National Academy of Sciences*, 47(1):110–112, 1961.
- [5] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [6] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.

A. Anhang

A.1. Zentrale Artefakte

Tabelle 6: Relevante Projektartefakte für die Reproduktion

Artefakt	Pfad im Repository
Experiment-Notebook	notebooks/01_experiment.ipynb
Ausgeführter finaler Lauf	models/runs/big_run_launcher_20260608_170138/
Finale Modelle	models/20260608_170145_*_agent.pkl
Checkpoints	models/checkpoints/<agent-seed>/
Finale Metriken	models/evaluations/20260610_164317/
Projektgrafiken	images/
Zusätzliche Berichtsauswertung	Doku/figures/

A.2. Ausgewählte Ergebnisse der Hyperparametersuche

Tabelle 7: Stufe C der Hyperparametersuche bei 20 Millionen Episoden

Agent	Konfiguration	Avg. Reward	Seed-Std.	Q-States
Baseline	c00_default	-0,04906	0,00138	280
Counting	c05_long_default	-0,05259	0,00052	108.681
Counting	c09_lr01_gamma098	-0,05353	0,00101	108.597
Counting	c00_default	-0,05442	0,00138	108.590
Counting	c15_long_high_lr	-0,05528	0,00106	108.479

A.3. Reproduktionshinweise

Das Experiment-Notebook erzeugt die finalen Evaluations-, True-Count- und Policy-Grafiken unter `images/` aus den gespeicherten Modellen und Evaluationsdaten. Für kompakte Berichts-darstellungen wurden die finalen Metriken und True-Count-Ergebnisse aus den gespeicherten CSV-Dateien zusammengefasst. Zusätzlich wurden die 60 Checkpoints mit jeweils 100.000 Episoden ausgewertet. Die resultierenden Zusammenfassungen liegen unter `Doku/figures/`. Der LaTeX-Bericht wird unter Windows mit `build-latex.ps1` kompiliert.